CETP canonical results

This file is generated by `make results`. Do not edit it, and do not quote any number about CETP from anywhere else. Every figure in the paper, the deck, the LLD and the README must come from here or from `results/canonical_metrics.json`.

- dataset: `data/corrected/tpch_dataset_corrected_v2.csv` — 5 machines × 21 queries × 20 repeats = 2100 rows → 420 ordered dev→prod pairs
- machines: c5a, c7i, m5a, r5n, z1d (**5**, all EBS-backed, SF1). Q15 excluded → 21 queries.
- environment: Python 3.13.2, scikit-learn 1.8.0, numpy 2.5.3, pandas 2.2.3; `random_state=42`, 2000 bootstrap resamples

Headline

Leave-one-machine-out, multi-core compute signal, classifier refit inside every fold:

> **analytical roofline 21.61% MAPE vs naive-linear 23.05%** — improvement +1.44pp, 95% cluster-bootstrap CI [0.32, 2.56], excludes zero (significant).

The qualification that must travel with the headline

`cmp_only` — scale by `1/compute_ratio` alone, with no classifier, no roofline weighting and no fitted parameters — scores **20.06% MAPE**, better than the analytical roofline's 21.61%. The paired difference is +1.55pp with CI [-0.41, 3.51], which **includes zero**.

> **The bottleneck classifier and roofline weighting are statistically indistinguishable from their own simplest component.** The gain over the bandwidth baseline is real, but it is attributable to using *compute* rather than *bandwidth* as the scaling signal — not to the classification or the weighting. Any claim that the roofline model is the contribution is unsupported by this evaluation, and `cmp_only` must be reported alongside it.

1. Scaling-factor accuracy

`q-error = max(pred/actual, actual/pred)`, the metric the database literature reports. A perfect model scores MAPE 0, MdAPE 0, q-error 1.000.

Multi-core aggregate compute signal (correct sign, r = −0.691)

LOMO: leave-one-machine-out — generalization to **new hardware**

approach	MAPE %	MdAPE %	q-error (median)	q-error (p95)
naive (prod = dev)	31.96	27.67	1.306	2.053
naive-linear (1/bandwidth_ratio)	23.05	19.37	1.220	1.670
cmp-only (1/compute_ratio)	20.06	15.87	1.170	1.701
analytical roofline	21.61	18.14	1.198	1.694
bottleneck-gated	22.32	19.04	1.207	1.657
learned quantile GBR (p50)	23.90	20.50	1.227	1.715

LOQO: leave-one-query-out — generalization to **new queries**

approach	MAPE %	MdAPE %	q-error (median)	q-error (p95)
naive (prod = dev)	31.96	27.67	1.306	2.053
naive-linear (1/bandwidth_ratio)	23.05	19.37	1.220	1.670
cmp-only (1/compute_ratio)	20.06	15.87	1.170	1.701
analytical roofline	21.66	17.80	1.198	1.700
bottleneck-gated	22.79	19.34	1.220	1.670
learned quantile GBR (p50)	17.29	12.05	1.127	1.661

LOMQO: leave-one-machine-**and**-query-out — both at once

approach	MAPE %	MdAPE %	q-error (median)	q-error (p95)
naive (prod = dev)	31.96	27.67	1.306	2.053
naive-linear (1/bandwidth_ratio)	23.05	19.37	1.220	1.670
cmp-only (1/compute_ratio)	20.06	15.87	1.170	1.701
analytical roofline	21.64	17.81	1.198	1.702
bottleneck-gated	22.79	19.34	1.220	1.670
learned quantile GBR (p50)	26.86	21.63	1.243	1.882

Single-core compute signal (wrong sign, r = +0.213)

LOMO: leave-one-machine-out — generalization to **new hardware**

approach	MAPE %	MdAPE %	q-error (median)	q-error (p95)
naive (prod = dev)	31.96	27.67	1.306	2.053
naive-linear (1/bandwidth_ratio)	23.05	19.37	1.220	1.670
cmp-only (1/compute_ratio)	51.44	37.57	1.448	2.743
analytical roofline	31.86	23.78	1.270	2.181
bottleneck-gated	28.40	21.21	1.236	2.203
learned quantile GBR (p50)	33.04	28.71	1.350	2.033

LOQO: leave-one-query-out — generalization to **new queries**

approach	MAPE %	MdAPE %	q-error (median)	q-error (p95)
naive (prod = dev)	31.96	27.67	1.306	2.053
naive-linear (1/bandwidth_ratio)	23.05	19.37	1.220	1.670
cmp-only (1/compute_ratio)	51.44	37.57	1.448	2.743
analytical roofline	27.30	24.44	1.292	1.768
bottleneck-gated	25.18	19.62	1.222	1.886
learned quantile GBR (p50)	18.28	12.86	1.132	1.608

LOMQO: leave-one-machine-**and**-query-out — both at once

approach	MAPE %	MdAPE %	q-error (median)	q-error (p95)
naive (prod = dev)	31.96	27.67	1.306	2.053
naive-linear (1/bandwidth_ratio)	23.05	19.37	1.220	1.670
cmp-only (1/compute_ratio)	51.44	37.57	1.448	2.743
analytical roofline	27.48	24.65	1.296	1.765
bottleneck-gated	25.18	19.62	1.222	1.886
learned quantile GBR (p50)	34.18	29.09	1.340	2.129

2. Leakage audit

Every prior LOMO script loaded one globally-fit `models/bottleneck_classifier.pkl` and applied it to held-out data. Since `p_compute/p_bandwidth/p_io/p_mixed` and `analytical_scaling` are features, that classifier's exposure to the held-out rows leaked into every reported number. Below: the old leaky protocol vs a classifier refit inside each fold. Positive inflation = the old number was optimistic.

protocol	approach	MAPE leaky	MAPE leakage-free	inflation pp
LOMO	naive	31.96	31.96	+0.00
LOMO	naive_linear	23.05	23.05	+0.00
LOMO	cmp_only	20.06	20.06	+0.00
LOMO	analytical	21.61	21.61	−0.00
LOMO	gated	22.32	22.32	+0.00
LOMO	learned	25.62	23.90	−1.72
LOQO	naive	31.96	31.96	+0.00
LOQO	naive_linear	23.05	23.05	+0.00
LOQO	cmp_only	20.06	20.06	+0.00
LOQO	analytical	21.61	21.66	+0.05
LOQO	gated	22.32	22.79	+0.48
LOQO	learned	17.04	17.29	+0.25
LOMQO	naive	31.96	31.96	+0.00
LOMQO	naive_linear	23.05	23.05	+0.00
LOMQO	cmp_only	20.06	20.06	+0.00
LOMQO	analytical	21.61	21.64	+0.02
LOMQO	gated	22.32	22.79	+0.48
LOMQO	learned	26.56	26.86	+0.30

Why LOMO is barely affected: on the corrected dataset all six classifier features (`cost`, `shared_hit`, `shared_read`, `total_buffers`, `io_ratio`, `rows`) are **identical across all five machines** for a given query — the buffer correction replaced each query's counts with a machine-invariant value, and `cost` / `rows` never varied by machine. So the held-out machine's classifier rows are exact duplicates of rows from the other four, and refitting without them changes almost nothing. The leak was real but, under LOMO, structurally harmless.

The consequence that does matter: six of the thirteen learned-model features (`dev_cost`, `dev_total_buffers`, `dev_shared_hit`, `dev_shared_read`, `dev_rows`, `dev_io_ratio`) are therefore **pure functions of** `query_id` — a one-hot encoding of the query in disguise. Under LOMO every query appears in training, so the GBR can memorise per-query scaling behaviour rather than learning from workload structure. That is why its LOQO and LOMQO numbers are the honest measure of what it has actually learned, and why they should be reported alongside LOMO rather than instead of it.

3. Per-held-out-machine breakdown (LOMO, multi-core)

Pooled MAPE hides the single most interesting result in this project, so it is broken out.

held-out machine	naive-linear	analytical	gated	learned GBR
c5a	19.68	19.80	19.55	28.86
c7i	27.75	25.39	26.67	31.37
m5a	20.58	20.36	20.26	19.53
r5n	25.97	22.64	24.41	22.45
z1d	21.26	19.87	20.69	17.27

`c5a` is the hardware outlier (highest bandwidth 14.9, lowest single-core compute). The learned model degrades sharply there while the formula-based estimators, which need no training example of a machine to use its measured ratios, degrade gracefully — the interpolation-not-extrapolation finding.

4. Per-bottleneck-class breakdown (LOMO, multi-core)

class	naive-linear	analytical	gated	learned GBR
bandwidth	17.64	17.61	17.64	21.10
compute	22.63	17.55	17.50	23.20
io	28.79	28.76	28.79	28.05
mixed	23.04	21.58	23.04	23.56

5. Bottleneck classifier

Leave-one-**query**-out, 21 folds. This is the only honest protocol: because the classifier's features are machine-invariant (see §2), leave-one-machine-out would test on exact duplicates of training rows.

- **macro-F1 = 0.375**
- trivial always-predict-`mixed` baseline = 0.161

class	precision	recall	F1	support
bandwidth	0.500	0.750	0.600	20
compute	1.000	0.333	0.500	15
io	0.000	0.000	0.000	20
mixed	0.400	0.400	0.400	50

Confusion matrix (rows = true, columns = predicted):

	bandwidth	compute	io	mixed
bandwidth	15	0	0	5
compute	5	5	0	5
io	0	0	0	20
mixed	10	0	20	20

The 0.562 figure that appears in older documents is wrong. It was computed on the buffer-bug dataset. On the corrected dataset the same protocol gives 0.375, and `io` recall is 0.000.

These labels are 21 hand-assigned judgement calls, not measurements (Q2 and Q13 are acknowledged coin flips), and because the features are machine-invariant the classifier is effectively a 21-row lookup table. Replacing the labels with measured resource elasticity is Tier 1 of `TASKS.md`.

6. Measurement noise

The collection protocol specified rejecting any instance with timing CV > 5%, but the metric was never carried into the dataset. It is now a stored column (`results/canonical_measurement_cv.csv`).

statistic	all 20 repetitions	warm 19 (used)
median CV	2.48%	0.54%
mean CV	9.68%	1.55%
max CV	119.98%	16.50%
cells over the 5% threshold	42 / 105	11 / 105

105 cells (machine × query), 20 repetitions each, **1 discarded as warmup** → 19 used.

Noisiest remaining cells: r5n/q6 (16.5%), z1d/q19 (9.2%), c7i/q19 (9.1%), c7i/q5 (7.3%), c7i/q8 (6.8%).

The cold-cache first repetition — a data-quality defect found by this audit

Carrying the CV metric into the dataset for the first time immediately exposed a problem the project had never recorded: **repetition 1 of each cell is a cold-cache run.**

observation	value
cells where repetition 1 is the slowest of 20	61 / 105
median repetition-1 slowdown vs the warm median	1.02x
worst repetition-1 slowdown	8.33x

Example — `z1d` / `q3` : 7612.3 ms, then nineteen runs between 911.8 and 916.3 ms. The cell's CV is 120.0% including that run and 0.1% without it.

What it contaminates, per cell, if the warmup run is kept:

target	cells inflated > 5%	worst inflation
p50	0 / 105	1.007x
p95	17 / 105	1.368x
p99	46 / 105	6.922x

The p50 headline is unaffected — with one outlier among twenty runs the median barely moves (worst case 0.7%), so every `scaling_factor_p50` result in this file stands whichever policy is used.

Every p95 and p99 result published before this audit is contaminated. At n=20 the 95th and 99th percentiles sit at the very top of the sorted sample — exactly where the cold-cache run lands. So those targets were substantially measuring first-run page-cache misses rather than warm tail latency. This is the most likely explanation for the previously reported quantile under-coverage (80.7% empirical at nominal 95%, 94.4% at nominal 99%): the models were being asked to predict a statistic that was itself an artifact. Any p95/p99 number, calibration table, or p99-based SLA-gate result from before this change must be regenerated.

7. Known inconsistency, not yet fixed

`models/hardware_signature.pkl` and `gate/cetp.yml` still carry the **old single-core** compute scores (c5a 0.593, c7i 0.807, …). The headline result requires the **multi-core** aggregate scores (c5a 40.81, c7i 25.89, …). **The deployed gate therefore does not embody the headline finding.** Fixing it means regenerating the shipped models and re-running the gate's 24 tests; tracked in `TASKS.md`.

8. What these numbers do and do not support

Supported: on 5 EBS-backed EC2 instance types at SF1, with 21 TPC-H queries, a roofline formula weighted by bottleneck probabilities and driven by a **multi-core** compute signal beats a pure bandwidth-ratio baseline by a small but bootstrap-significant margin, and formula-based estimators extrapolate to unseen hardware better than the learned model.

Not supported: any claim about hardware outside these 5 instance types, about non-EBS storage, about scale factors other than SF1, about concurrent workloads, about queries outside these 21, or about how *often* a new machine behaves like the c5a outlier (n=1). The `io` bottleneck class reflects buffer-touch patterns under a single uniform storage backend, not storage diversity.